Common Problems

Inventory Management

By Evan King · Published Dec 18, 2025 · hard

Understanding the Problem

📦 **What is an Inventory Management System?**

An inventory management system tracks product stock across multiple warehouse locations. When inventory arrives, the system records it. When orders ship, the system deducts stock. The system can also transfer inventory between locations and alert managers when stock runs low.

Requirements

When the interview starts, you'll get something like this:

> "Design an inventory management system that tracks products across multiple warehouses. The system needs to handle adding and removing inventory, transferring stock between locations, and alerting when inventory runs low."

That's a start, but there's a lot of room for interpretation. Before touching the whiteboard, spend a few minutes clarifying what you're actually building.

Clarifying Questions

Structure your questions around four themes: what operations does the system support, what can go wrong, what's in scope, and what might we extend later?

Here's how the conversation might go:

> **You:** "When you say 'multiple warehouses,' are we talking about a fixed set of warehouses configured at startup, or can warehouses be added dynamically?"
> **Interviewer:** "Keep it simple. Fixed set of warehouses. You can assume they're configured when the system initializes."

Good. We're not building warehouse lifecycle management.

> **You:** "For the low-stock alerts you mentioned, how do those work? Do we configure a threshold per product, or is it more granular?"
> **Interviewer:** "It should be per product per warehouse. Different warehouses might need different thresholds for the same product. When stock drops below the threshold, trigger a notification."

That's important. Alerts are warehouse-specific, not global. A product could be low in Warehouse A but fine in Warehouse B.

> **You:** "How should the notification happen? Are we sending emails, calling a webhook, or just returning something to the caller?"
> **Interviewer:** "Good question. Let's keep it pluggable. The system should call some callback interface when stock is low. What happens after that - email, webhook, logging - is someone else's problem."

Perfect. We're building the alert mechanism, not the notification delivery.

> **You:** "What about invalid operations? Should we allow negative inventory, or reject operations that would take stock below zero?"
> **Interviewer:** "Reject them. If someone tries to remove 100 units but we only have 50, that should fail. Same with transfers, validate before moving anything."

The system enforces invariants. Stock can't go negative.

> **You:** "What about concurrent access? If two processes are trying to modify the same warehouse's inventory at the same time, do we need to handle that?"
> **Interviewer:** "Yes, concurrency is important here. Multiple operations could be happening simultaneously - one warehouse receiving a shipment while another is fulfilling an order for the same product. Make sure operations are thread-safe."

Good. We need to think about synchronization from the start.

> **You:** "Last one. What's out of scope? Are we managing product catalogs, handling orders, that kind of thing?"
> **Interviewer:** "Yes. Products exist externally. Orders and payments are handled upstream. Keep the focus on the inventory tracking logic."

Final Requirements

After that back-and-forth, you'd write this on the whiteboard:

FINAL REQUIREMENTS

```
Requirements:
1. Track inventory for products across multiple warehouses
2. Add stock to a specific warehouse (receiving shipments)
3. Remove stock from a specific warehouse (fulfilling orders)
4. Check availability: given a product and quantity, return which warehouses can f
5. Transfer stock between warehouses
6. Low-stock alerts
7. Reject operations that would result in negative inventory
8. System must be thread-safe to handle concurrent operations

Out of Scope:
- Product catalog management (products exist externally)
- Order processing / payment / serviceability
- Persistence
```

Perfect. We've scoped out the problem and have concrete requirements to work from. The next step is identifying what objects we need to build this system.

Core Entities and Relationships

Now that the requirements are clear, we need to figure out what objects make up the system. The trick is scanning the requirements for nouns that represent things with behavior or state. We start by considering each of the nouns in our requirements as candidates, pruning until we have a list of entities that make sense to model.

It's helpful to apply a simple filter: if something maintains changing state or enforces rules, it likely deserves to be its own entity. If it's just information attached to something else, it's probably just a field on another class.

The candidates are:

Product - The product catalog lives outside our system. We just need to know "how many units of product X are in warehouse Y." Product is a key (string or integer) that identifies what we're counting, not a class with behavior.

Warehouse - This is definitely an entity. A warehouse holds inventory for multiple products, tracks how many units of each product it has, and knows when to fire alerts. It needs to enforce the "no negative stock" rule to prevent operations that would leave quantities below zero. It also needs to check its alert configurations after every stock change and fire notifications when thresholds are crossed. Clear state and behavior.

InventoryManager - Something needs to orchestrate the whole system. When someone says "transfer 50 units of product X from Warehouse A to Warehouse B," something needs to look up both warehouses, validate the operation, and coordinate the movement. When someone asks "which warehouses can fulfill an order for 100 units of product Y?", something needs to query all warehouses and aggregate the results. That's the manager. It's the entry point for all operations and owns the collection of warehouses.

AlertConfig - When we configure a low-stock alert, we're defining two pieces of information: a threshold (when should we alert?) and a callback (what should we do?). This combination is worth modeling as a small value object rather than storing the pieces separately. It keeps the relationship explicit and makes the code easier to reason about.

AlertListener - The "what to do when stock is low" part. This should be an interface so different implementations can handle notifications in different ways. One implementation might send email. Another might call a webhook. A third might just log to a file. The inventory system doesn't know or care which notification mechanism gets used. It just calls the interface method when stock drops below a threshold.

After filtering, we're left with four entities:

Entity	Responsibility
InventoryManager	The orchestrator. Owns all warehouses and coordinates operations across them. When you want to transfer stock, check availability across multiple locations, or configure alerts, you go through this class. It's the only public API for the system.
Warehouse	Represents a single storage location. Holds a map of productId → quantity. Enforces the "no negative stock" invariant. Owns its alert configurations and fires alerts when stock changes trigger thresholds. Doesn't know about other warehouses.

Entity	Responsibility
AlertConfig	A value object grouping the threshold and the listener to notify when stock drops below that threshold.
AlertListener	An interface defining the callback contract. Implementations receive warehouse ID, product ID, and current quantity when stock drops below a threshold. This decouples "stock is low" from "what to do about it."

The relationships between these entities follow a clear hierarchy. InventoryManager sits at the top, holding a map from warehouse ID strings to Warehouse objects. This gives it the ability to route operations to the correct warehouse based on the ID provided by external callers.

Class Design

With our entities identified, it's time to define their interfaces. What state does each one hold, and what methods does it expose?

We'll work top-down, starting with `InventoryManager` since it's the entry point, then drilling into `Warehouse`, `AlertConfig`, and `AlertListener`.

For each class, we'll trace back to the requirements and ask two questions: what does this entity need to remember, and what operations does it need to support?

InventoryManager

InventoryManager is the **facade** that external code interacts with. Let's derive its state from requirements:

Requirement	What `InventoryManager` must track
"Track inventory for products across multiple warehouses"	The collection of all warehouses
"Add stock to a specific warehouse"	Need to look up warehouses by ID
"Transfer stock between warehouses"	Need references to both source and destination warehouses

This gives us:

INVENTORYMANAGER STATE

```
class InventoryManager:
    - warehouses: Map<string, Warehouse>
```

Why `warehouses` as a map. This field maps warehouse ID strings (like "WAREHOUSE_EAST" or "DC_01") to their corresponding Warehouse objects. When a caller says `addStock("WAREHOUSE_EAST", "PROD_123", 50)`, we need to find the East warehouse quickly. The map gives us O(1) lookup by ID. The alternative would be a list that we scan every time, which works but makes the intent less clear. The map structure explicitly represents "warehouse IDs map to warehouse objects."

Why warehouse IDs are strings, not integers. IDs could technically be integers, but strings are more flexible for real systems. You can use meaningful identifiers like "WAREHOUSE_CALIFORNIA" or "DC_NY_01" instead of arbitrary numbers. In production, these IDs often come from external systems or databases where they're already strings. For interview scope, either choice is defensible - just be consistent.

Now for operations. Every method on InventoryManager should correspond to a concrete need from the requirements. Let's trace each one:

Need from requirements	Method on `InventoryManager`
"Add stock to a specific warehouse"	`addStock(warehouseId, productId, quantity)`
"Remove stock from a specific warehouse"	`removeStock(warehouseId, productId, quantity)` returns boolean for success/failure
"Check availability across warehouses"	`getWarehousesWithAvailability(productId, quantity)` returns list of warehouse IDs
"Transfer stock between warehouses"	`transfer(productId, fromWarehouseId, toWarehouseId, quantity)` returns boolean
"Configure low-stock alerts"	`setLowStockAlert(warehouseId, productId, threshold, listener)`

Putting it together:

INVENTORYMANAGER

```
class InventoryManager:
    - warehouses: Map<string, Warehouse>

    + InventoryManager(warehouseIds)
    + addStock(warehouseId, productId, quantity) -> void
    + removeStock(warehouseId, productId, quantity) -> boolean
    + transfer(productId, fromWarehouseId, toWarehouseId, quantity) -> boolean
    + getWarehousesWithAvailability(productId, quantity) -> List<string>
    + setLowStockAlert(warehouseId, productId, threshold, listener) -> void
```

The constructor takes a list of warehouse IDs and creates a Warehouse instance for each one, storing them in the map. This happens once at system initialization. Since our requirements say warehouses are "fixed at startup," we don't need methods to add or remove warehouses dynamically. All the warehouse objects get created upfront.

The rest of the methods form the main API for inventory operations:

`addStock` is straightforward - look up the warehouse and tell it to add quantity. This operation always succeeds (you can always receive more inventory), so there's no return value. It returns void.

`removeStock` looks up the warehouse and asks it to remove quantity. But this operation can fail if there isn't enough stock. The return value (boolean) tells the caller whether the operation succeeded. This asymmetry with `addStock` is intentional because add always works, whereas remove sometimes doesn't.

`transfer` is more interesting. It needs to find both the source warehouse and the destination warehouse, validate that the source has enough stock, then coordinate moving it. This operation can also fail (insufficient stock, invalid warehouse IDs), so it returns a boolean.

`getWarehousesWithAvailability` answers the question "which warehouses can fulfill this quantity?" It iterates through all warehouses, checking each one, and returns a list of warehouse IDs that have enough stock. This aggregates information across the entire system.

`setLowStockAlert` delegates alert configuration to a specific warehouse. The manager doesn't manage alerts itself, it just routes the request to the right warehouse, which then owns that alert configuration.

> Why not manage alerts in the `InventoryManager`? Because the manager doesn't know about specific products. It only knows about the collection of warehouses and the operations it can perform on them. The manager is the public API for the system, so it needs to be simple and focused. The Warehouse class is where the actual inventory management happens, so it makes sense for it to own its alert configurations.

Warehouse

Warehouse represents one physical storage location. It's where the actual inventory tracking happens. It knows how many units of each product are in stock, enforces the "no negative inventory" constraint, and fires alerts when stock drops below configured thresholds.

Let's derive its state from requirements:

Requirement	What `Warehouse` must track
"Track inventory for products"	Map from product ID to quantity
"Low-stock alerts per product per warehouse"	Alert configurations for each product
"Multiple warehouses"	Its own ID to distinguish itself

That gives us the following state:

WAREHOUSE STATE

```
class Warehouse:
    - id: string
    - inventory: Map<string, int>
    - alertConfigs: Map<string, List<AlertConfig>>
```

Why `id` is needed. When an alert fires and calls `listener.onLowStock(warehouseId, productId, currentQuantity)`, the listener needs to know which warehouse is reporting low stock. The warehouse needs to remember its own ID so it can include it in alert callbacks. Without this, you'd have no way to tell whether "product X is low" refers to the California warehouse or the New York warehouse.

Why `inventory` is a map from product ID to quantity. This is the core data structure for tracking stock. Each product ID points to a single integer representing how many units we have. The map means we can check or update any product's quantity in O(1) time. Products not in the map implicitly have zero stock. We don't need to initialize entries for products we've never received.

Why `alertConfigs` maps product IDs to lists of AlertConfig objects. A product can have multiple alert configurations with different thresholds and different listeners. Maybe you want one alert at 50 units to send an email saying "order more soon," and another alert at 10 units to page someone saying "critical shortage." Storing a list of configurations per product supports this naturally. When stock changes, we iterate through all configs for that product and check each threshold. Most products will have zero or one alert config, but the list structure supports multiple without extra complexity.

> You might wonder whether to support a single alert per product or multiple alerts. This is worth discussing with your interviewer. We're going with multiple because you might want a warning at 20 units, an urgent alert at 5 units, and a critical alert at 0. Different thresholds can trigger different notification channels or go to different teams. The implementation is straightforward either way. One alert means a simple field lookup, multiple alerts means iterating through a list to find matches.

For operations, we need methods to support all the inventory operations plus alert configuration:

Need from requirements	Method on `Warehouse`
"Add stock to a specific warehouse"	`addStock(productId, quantity)`
"Remove stock from a specific warehouse"	`removeStock(productId, quantity)` returns boolean

Need from requirements	Method on `Warehouse`
"Check if warehouse can fulfill quantity"	`checkAvailability(productId, quantity)` returns boolean
"Query current stock level"	`getStock(productId)` returns int
"Configure alerts"	`setLowStockAlert(productId, threshold, listener)`

The only thing we're missing is a way to fire alerts when stock changes. For this, we need a private helper method.

Internal need	Method on `Warehouse`
Collect alerts when stock crosses thresholds	`getAlertsToFire(productId, previousQty, newQty)` private method

The first four methods handle inventory operations. `addStock` increases quantity, `removeStock` decreases it (and can fail if there's not enough stock), `checkAvailability` answers "can I fulfill this quantity?", and `getStock` returns the current level for reporting or debugging.

`setLowStockAlert` registers a new alert configuration. It takes a threshold and a listener, wraps them in an AlertConfig object, and adds it to the list for that product. If this product already has other alert configs, the new one gets appended to the list. If this is the first alert for this product, we create a new list with just this config.

`getAlertsToFire` is the private helper that runs after every `addStock` or `removeStock` operation. It takes the product ID, the previous quantity (before the change), and the new quantity (after the change). It checks the alert configs for this product to see if any thresholds were crossed, and returns the list of listeners that should be notified. The caller then fires those listeners outside the synchronized block to avoid holding locks during I/O.

WAREHOUSE

```
class Warehouse:
    - id: string
```

```
    - inventory: Map<string, int>
    - alertConfigs: Map<string, List<AlertConfig>>

    + Warehouse(id)
    + addStock(productId, quantity) -> void
    + removeStock(productId, quantity) -> boolean
    + getStock(productId) -> int
    + checkAvailability(productId, quantity) -> boolean
    + setLowStockAlert(productId, threshold, listener) -> void
    - getAlertsToFire(productId, previousQty, newQty) -> List<AlertListener>
```

AlertConfig

AlertConfig is a compact value object that pairs a threshold with a listener. The listener is an object that registers interest in events and gets notified when those events happen. The warehouse doesn't need to know what the listener does when notified. It just calls a method on the listener when stock crosses a threshold and lets the listener handle the notification.

ALERTCONFIG

```
class AlertConfig:
    - threshold: int
    - listener: AlertListener

    + AlertConfig(threshold, listener)
    + getThreshold() -> int
    + getListener() -> AlertListener
```

The constructor takes the threshold and listener. The getters expose the configuration to the Warehouse for alert checking logic.

> AlertConfig doesn't need to be a class. Use whatever fits your language - a struct in Go, a dataclass in Python, a simple interface in TypeScript. The point is keeping threshold and listener paired in a single type instead of managing parallel data structures.

The value of modeling this as a separate class (or struct, or type) is clarity. When you see `List<AlertConfig>` in the Warehouse class, you immediately understand "this is a list of alert

configurations." If instead we had `Map<int, AlertListener>` (threshold to listener), the relationship is less obvious. Is the key really a threshold, or could it be something else? The named type makes the intent explicit.

AlertListener

AlertListener is an interface that defines the contract for handling low-stock notifications. It's what makes the alert system pluggable:

ALERTLISTENER

```
interface AlertListener:
    + onLowStock(warehouseId, productId, currentQuantity) -> void
```

That's the complete interface. One method. When stock drops below a threshold, the warehouse calls this method with three pieces of context: which warehouse is reporting the alert, which product is low, and what the current quantity is. The listener can use this information however it wants.

Different implementations can handle notifications in completely different ways:

EXAMPLE IMPLEMENTATIONS

```
class EmailAlertListener implements AlertListener:
    onLowStock(warehouseId, productId, currentQuantity)
        sendEmail("Stock alert: " + productId + " is low in " + warehouseId)

class WebhookAlertListener implements AlertListener:
    onLowStock(warehouseId, productId, currentQuantity)
        httpPost("/alerts", {warehouse: warehouseId, product: productId, qty: curr

class LoggingAlertListener implements AlertListener:
    onLowStock(warehouseId, productId, currentQuantity)
        log("WARNING: " + warehouseId + " has only " + currentQuantity + " of " +
```

The inventory system doesn't know or care which implementation gets used. It just calls `listener.onLowStock()` when the threshold is crossed. The actual notification mechanism (email, webhook, logging, SMS, pushing to a queue, whatever) lives in the implementation. This

is the **Observer pattern** in action. Warehouses notify listeners when interesting events happen, and listeners decide what to do about it.

> In modern languages, simple observers are often implemented as callbacks or function references rather than interface implementations. JavaScript uses function callbacks, Python uses lambdas or function references, and Go uses function types. The interface approach shown here is useful for LLD interviews because it makes the structure explicit and works across all languages, but in production code you'd likely use whatever callback mechanism your language provides.

This design follows the **Dependency Inversion Principle**. The Warehouse (high-level inventory logic) doesn't depend on EmailSender or WebhookClient (low-level notification mechanisms). Both depend on the AlertListener interface (abstraction). You can swap notification implementations without touching the warehouse code at all.

In an interview, you typically wouldn't implement these concrete listener classes unless specifically asked. The important thing is defining the interface and explaining how different implementations would work. That demonstrates you understand polymorphism and the value of designing to interfaces rather than concrete classes.

Final Class Design

That's the complete class design. We have four pieces working together with clean **separation of concerns**:

InventoryManager sits at the top as the orchestrator, providing the public API and routing operations to the appropriate warehouse. It answers questions that require looking across multiple warehouses (like "which warehouses can fulfill this order?") and coordinates multi-warehouse operations (like transfers).

Warehouse handles the actual inventory tracking for one location. It maintains the quantity map, enforces the "no negative stock" constraint, manages its alert configurations, and fires notifications when thresholds are crossed. It doesn't know about other warehouses—it just manages its own stock.

AlertConfig keeps the threshold and listener together as a single configuration unit.

AlertListener defines the notification contract. Implementations handle the actual notification delivery—email, webhook, logging, whatever. The inventory system just calls the interface method when stock is low.

FINAL CLASS DESIGN

```
class InventoryManager:
    - warehouses: Map<string, Warehouse>

    + InventoryManager(warehouseIds)
    + addStock(warehouseId, productId, quantity) -> void
    + removeStock(warehouseId, productId, quantity) -> boolean
    + transfer(productId, fromWarehouseId, toWarehouseId, quantity) -> boolean
    + getWarehousesWithAvailability(productId, quantity) -> List<string>
    + setLowStockAlert(warehouseId, productId, threshold, listener) -> void

class Warehouse:
    - id: string
    - inventory: Map<string, int>
    - alertConfigs: Map<string, List<AlertConfig>>

    + Warehouse(id)
    + addStock(productId, quantity) -> void
    + removeStock(productId, quantity) -> boolean
    + getStock(productId) -> int
    + checkAvailability(productId, quantity) -> boolean
    + setLowStockAlert(productId, threshold, listener) -> void
    - getAlertsToFire(productId, previousQty, newQty) -> List<AlertListener>

class AlertConfig:
    - threshold: int
    - listener: AlertListener

    + AlertConfig(threshold, listener)
    + getThreshold() -> int
    + getListener() -> AlertListener

class AlertListener:
    + onLowStock(warehouseId, productId, currentQuantity) -> void
```

Each piece has a single, well-defined responsibility and the boundaries between them are clear. Now, we can move onto the implementation.

Implementation

With the class design locked in, it's time to actually implement the methods. We'll write pseudocode like is the case in most LLD interviews, but their is a **complete implementation** in common languages for reference at the end of this section.

For each method, we'll use this approach:

1. **Define the core logic** - The happy path when everything works correctly
2. **Handle edge cases** - Invalid inputs, boundary conditions, operations that can fail

The most interesting implementations are `InventoryManager.transfer()` (coordinating operations across warehouses) and `Warehouse.getAlertsToFire()` (the alert collection logic).

InventoryManager

InventoryManager is the orchestrator, so its methods are mostly lookups and delegation. Let's start with the constructor:

INVENTORYMANAGER CONSTRUCTOR

```
InventoryManager(warehouseIds)
    warehouses = {}
    for id in warehouseIds
        warehouses[id] = Warehouse(id)
```

The constructor takes a list of warehouse ID strings and creates a Warehouse instance for each one. All warehouses start empty with no inventory and no alerts configured. Since warehouses are fixed at startup (per our requirements), we don't need methods to add or remove warehouses later.

Now the operational methods. Let's start with the simple ones:

INVENTORYMANAGER.ADDSTOCK

```
addStock(warehouseId, productId, quantity)
    warehouse = warehouses[warehouseId]
    if warehouse == null
        throw error
```

```
    warehouse.addStock(productId, quantity)
```

Look up the warehouse and tell it to add stock. If the warehouse ID doesn't exist, fail fast with an error. This operation always succeeds once we've validated the warehouse (you can always receive more inventory).

INVENTORYMANAGER.REMOVESTOCK

```
removeStock(warehouseId, productId, quantity)
    warehouse = warehouses[warehouseId]
    if warehouse == null
        return false

    return warehouse.removeStock(productId, quantity)
```

Similar structure, but `removeStock` can fail if there's insufficient inventory. We return the boolean that the warehouse gives us, which tells the caller whether the operation succeeded.

INVENTORYMANAGER.GETWAREHOUSESWITHAVAILABILITY

```
getWarehousesWithAvailability(productId, quantity)
    available = []
    for id in warehouses.keys()
        warehouse = warehouses[id]
        if warehouse.checkAvailability(productId, quantity)
            available.add(id)
    return available
```

This method answers "which warehouses can fulfill this quantity?" We iterate through all warehouses, ask each one if it has enough stock, and collect the IDs of those that do. The result is a list of warehouse IDs (could be empty if no warehouse has sufficient stock).

INVENTORYMANAGER.SETLOWSTOCKALERT

```
setLowStockAlert(warehouseId, productId, threshold, listener)
    warehouse = warehouses[warehouseId]
    if warehouse == null
        throw error
```

```
warehouse.setLowStockAlert(productId, threshold, listener)
```

Alert configuration is warehouse-specific, so we just look up the warehouse and delegate. The warehouse owns its alert configurations.

Now the interesting one, `transfer`. This is where we coordinate operations across two warehouses. We'll model transfers as atomic for interview scope. In production you'd track in-transit state during shipment, which we'll cover in the **extensibility section**.

Core logic:

1. Validate the quantity is positive
2. Look up both the source and destination warehouses
3. Check that the source has enough stock
4. Remove from source, add to destination

Edge cases:

- Negative or zero quantity (reject)
- Invalid warehouse IDs (one or both don't exist)
- Insufficient stock in source warehouse

Sounds simple, but there's a problem. We need the transfer to be atomic.

An operation is atomic when it appears to happen all at once, with no observable in-between state. Either the entire operation completes successfully, or nothing happens at all. For transfers, atomic means stock disappears from the source and appears at the destination as a single indivisible step. No thread should ever see a state where the stock is missing from both warehouses, or where it exists in both places.

Without atomicity, weird things happen. Between removing stock from the source and adding it to the destination, other threads might observe intermediate states (stock temporarily vanishes from the system), or race conditions might create phantom inventory (stock appears in multiple places).

Let's see how this can go wrong and how to fix it.

Bad Solution: Check availability then transfer

Good Solution: Check return value from removeStock

Great Solution: Lock both warehouses (atomic transfer)

Most production systems use the "great" approach if they need strict inventory consistency, or the "good" approach if brief intermediate states are acceptable and they want better concurrency. The "bad" approach is never acceptable in production given that it has a correctness bug, not just a performance tradeoff.

Warehouse

Warehouse is where the real inventory management happens. It maintains the quantity map, enforces the "no negative stock" constraint, manages its alert configurations, and fires notifications when thresholds are crossed.

Before diving into the methods, we need to decide how alert firing should work. This choice affects the implementation of `checkAndFireAlerts`, which gets called from both `addStock` and `removeStock`. When exactly should alerts fire? There are a few ways to handle this, each with different behavior in production.

Bad Solution: Fire alert every time stock is below threshold

Great Solution: Fire alert only when crossing threshold from above

We'll implement the threshold-crossing approach. It handles both duplicate prevention and resettability with a simple condition check, no state tracking needed.

The constructor initializes the three fields:

WAREHOUSE CONSTRUCTOR

```
Warehouse(id)
    this.id = id
    inventory = {}
    alertConfigs = {}
```

All warehouses start with empty inventory and no alert configurations. Products get added to the maps on-demand as operations reference them.

Now `addStock`:

Before we write the method, we need to deal with concurrency. Multiple operations can hit the same warehouse simultaneously, so we must ensure the read-modify-write sequence stays atomic. Here are the main approaches:

Bad Solution: Ignore synchronization

Great Solution: Lock the whole warehouse map

Great Solution: Lock per product

We'll go with the coarse-grained lock because it's the clearest and perfectly adequate for a single in-memory warehouse. Here's the synchronized version:

WAREHOUSE.ADDSTOCK (NAIVE VERSION)

```
addStock(productId, quantity)
    synchronized(this)
        if quantity <= 0
            throw error

        currentQty = inventory[productId]
        if currentQty == null
            currentQty = 0

        newQty = currentQty + quantity
        inventory[productId] = newQty
```

```
        checkAndFireAlerts(productId, currentQty, newQty)
```

We validate the quantity is positive, get the current quantity (defaulting to 0 if this product hasn't been seen before), calculate the new quantity, update the map, then check and fire alerts.

However, there's a critical problem with calling `checkAndFireAlerts` inside the synchronized block. The listener callbacks might do network I/O (sending an email, calling a webhook), which can take seconds. While the listener executes, the warehouse lock is held, blocking all other operations. Worse, if the listener tries to call back into the warehouse (to check current stock for a richer alert message), you get a deadlock.

The fix is to separate "deciding which alerts to fire" from "firing them." Collect alerts while holding the lock, then invoke callbacks after releasing it:

WAREHOUSE.ADDSTOCK (CORRECTED)

```
addStock(productId, quantity)
    alertsToFire = null

    synchronized(this)
        if quantity <= 0
            throw error

        currentQty = inventory[productId]
        if currentQty == null
            currentQty = 0

        newQty = currentQty + quantity
        inventory[productId] = newQty

        alertsToFire = getAlertsToFire(productId, currentQty, newQty)

    // Fire alerts outside the synchronized block
    if alertsToFire != null
        for alert in alertsToFire
            alert.listener.onLowStock(id, alert.productId, alert.quantity)
```

The critical section only does fast in-memory operations. The slow part (invoking listeners) happens outside the lock. This pattern appears throughout concurrent systems: capture events under lock, dispatch notifications outside the lock.

> Don't forget to check alerts in `addStock`, not just `removeStock`. Stock can cross thresholds in both directions. If the threshold is 10 and stock goes from 5 to 12, no alert should fire (we crossed upward). The resettable approach captures that by comparing the previous and new quantities, resetting the flag when stock climbs back above the threshold.

Now `removeStock`:

WAREHOUSE.REMOVESTOCK

```
removeStock(productId, quantity)
    alertsToFire = null

    synchronized(this)
        if quantity <= 0
            return false

        currentQty = inventory[productId]
        if currentQty == null
            currentQty = 0

        if currentQty < quantity
            return false

        newQty = currentQty - quantity
        inventory[productId] = newQty

        alertsToFire = getAlertsToFire(productId, currentQty, newQty)

    // Fire alerts outside the synchronized block
    if alertsToFire != null
        for alert in alertsToFire
            alert.listener.onLowStock(id, alert.productId, alert.quantity)

    return true
```

This one's similar to `addStock` but includes the important check of "do we have enough stock?". If someone tries to remove 100 units but we only have 50, we return false without changing anything. This enforces our "no negative inventory" invariant. Only if we have sufficient stock do we proceed with the update, collect alerts, then fire them after releasing the lock.

> All warehouse mutations must be synchronized. Notice the `synchronized(this)` wrapper on both `addStock` and `removeStock`. For the same race-condition reasons discussed earlier, every public method that touches `inventory` or `alertConfigs` must acquire the same lock. This includes `setLowStockAlert` below.

As for the simple query methods, they're straightforward lookups:

WAREHOUSE.GETSTOCK

```
getStock(productId)
    synchronized(this)
        qty = inventory[productId]
        if qty == null
            return 0
        return qty
```

WAREHOUSE.CHECKAVAILABILITY

```
checkAvailability(productId, quantity)
    synchronized(this)
        if quantity <= 0
            return false

        currentQty = inventory[productId]
        if currentQty == null
            currentQty = 0
        return currentQty >= quantity
```

`getStock` returns the current quantity or 0 if the product hasn't been seen.

`checkAvailability` validates that the requested quantity is positive (checking for 0 or negative

quantities doesn't make sense), then checks if we have at least that amount. Both are straightforward lookups that directly access the inventory map within the synchronized block.

> Should read-only methods be synchronized too? Yes, to ensure they see consistent state. The safe choice is synchronizing all public methods that touch shared state, both reads and writes. Some implementations use more sophisticated patterns like read-write locks or volatile fields, but for interview scope, consistent locking is clearest. If you're doing this in production with extreme read-heavy workloads, you might optimize differently—but mention that tradeoff explicitly.

Now, we need to be able to set a new alert.

WAREHOUSE.SETLOWSTOCKALERT

```
setLowStockAlert(productId, threshold, listener)
    synchronized(this)
        if threshold <= 0
            throw error

        if listener == null
            throw error

        configs = alertConfigs[productId]
        if configs == null
            configs = []
            alertConfigs[productId] = configs

        config = AlertConfig(threshold, listener)
        configs.add(config)
```

We validate the threshold is positive (a threshold of 0 or negative doesn't make sense for low-stock alerts) and the listener is not null (we need something to notify). Then we get the list of alert configurations for this product (creating an empty list if this is the first alert), create a new AlertConfig, and add it to the list. Multiple alerts per product are supported naturally by the list structure. Like all warehouse mutations, this must be synchronized to prevent concurrent modifications to the `alertConfigs` map.

Now for the `getAlertsToFire` method. As we decided in the deep dive earlier, we check if stock crossed the threshold from above to below:

WAREHOUSE.GETALERTSTOFIRE

```
getAlertsToFire(productId, previousQty, newQty)
    configs = alertConfigs[productId]
    if configs == null
        return null

    alertsToFire = []

    for config in configs
        if previousQty >= config.threshold && newQty < config.threshold
            alertsToFire.add({config.listener, productId, newQty})

    if alertsToFire.isEmpty()
        return null

    return alertsToFire
```

We iterate through all alert configurations for this product. For each config, we check if stock crossed the threshold downward (was at or above, now below). If so, we collect the alert details.

This method only builds a list - it doesn't invoke any callbacks. The caller (shown in `addStock` and `removeStock` above) fires the alerts after releasing the lock.

AlertConfig

AlertConfig is a lightweight data holder pairing a threshold with a listener:

ALERTCONFIG

```
class AlertConfig:
    threshold: int
    listener: AlertListener

    AlertConfig(threshold, listener)
        this.threshold = threshold
        this.listener = listener
```

```
    getThreshold()
        return threshold

    getListener()
        return listener
```

Both fields are immutable after construction. The crossing check in `getAlertsToFire` handles duplicate prevention and resettability without any state on the config itself.

AlertListener

AlertListener is an interface, so we just define the contract:

ALERTLISTENER

```
interface AlertListener:
    onLowStock(warehouseId, productId, currentQuantity)
```

In an interview, you'd typically stop here after explaining how different implementations would work. But to make it concrete, here is one example implementation:

EMAILALERTLISTENER

```
class EmailAlertListener implements AlertListener:
    emailAddress: string

    EmailAlertListener(emailAddress)
        this.emailAddress = emailAddress

    onLowStock(warehouseId, productId, currentQuantity)
        subject = "Low stock alert: " + warehouseId
        body = productId + " is low (" + currentQuantity + " units remaining)"
        sendEmail(emailAddress, subject, body)
```

Each implementation handles notifications differently, but they all follow the same contract. The inventory system doesn't care which one you plug in, it just calls `onLowStock` when thresholds are crossed.

Complete Code Implementation

While most companies only require pseudocode during interviews, some do ask for full implementations of at least a subset of the classes or methods. Below is a complete working implementation in common languages for reference.

InventoryManager.py | Warehouse.py | AlertConfig.py | Alert | Python

```python
from typing import Optional


class InventoryManager:
    def __init__(self, warehouse_ids: list[str]):
        self._warehouses: dict[str, Warehouse] = {}
        for warehouse_id in warehouse_ids:
            self._warehouses[warehouse_id] = Warehouse(warehouse_id)

    def add_stock(self, warehouse_id: str, product_id: str, quantity: int) -> None
        warehouse = self._warehouses.get(warehouse_id)
        if warehouse is None:
            raise ValueError(f"Warehouse {warehouse_id} not found")

        warehouse.add_stock(product_id, quantity)

    def remove_stock(self, warehouse_id: str, product_id: str, quantity: int) -> b
        warehouse = self._warehouses.get(warehouse_id)
        if warehouse is None:
            return False

        return warehouse.remove_stock(product_id, quantity)

    def transfer(
        self,
        product_id: str,
        from_warehouse_id: str,
        to_warehouse_id: str,
```

Verification

After implementing the methods, you want to walk through a few test cases to show your implementation handles both happy paths and edge cases correctly. These are the top 3 things I'd test:

Test Case 1: Alert Threshold Crossing Behavior

When testing the alert logic, we're verifying that alerts fire when crossing the threshold downward, don't fire duplicates while stock stays low, and fire again if stock recovers and then drops again.

Setup: Warehouse with 15 widgets, alert threshold at 10

```
Initial: inventory["WIDGET"] = 15

Operation: removeStock("WIDGET", 6)
  previousQty = 15
  newQty = 15 - 6 = 9
  getAlertsToFire(15, 9):
    - previousQty=15 >= threshold=10 AND newQty=9 < threshold=10
    - Crossed threshold downward! Fire alert
  Result: Alert fires

Operation: removeStock("WIDGET", 2)
  previousQty = 9
  newQty = 9 - 2 = 7
  getAlertsToFire(9, 7):
    - previousQty=9 < threshold=10, condition is false
    - No crossing occurred (already below threshold)
  Result: No alert (no duplicate)

Operation: addStock("WIDGET", 15)
  previousQty = 7
  newQty = 7 + 15 = 22
  getAlertsToFire(7, 22):
    - previousQty=7 < threshold=10, condition is false
    - No downward crossing (stock is rising)
  Result: No alert (stock recovering)

Operation: removeStock("WIDGET", 13)
  previousQty = 22
  newQty = 22 - 13 = 9
  getAlertsToFire(22, 9):
    - previousQty=22 >= threshold=10 AND newQty=9 < threshold=10
    - Crossed threshold downward again! Fire alert
  Result: Alert fires (naturally reset by recovery)
```

This confirms the crossing check works. We get an alert when stock first drops below 10, no duplicates while it stays low, and another alert fires if stock recovers above 10 and then drops below again. No state tracking needed.

Test Case 2: Atomic Transfer With Proper Locking

When testing transfers, we're wanting to verify that the operation is atomic across both warehouses - no other thread sees an intermediate state where inventory is missing from both.

Setup: EAST has 7 widgets, WEST has 0 widgets

```
Operation: transfer("WIDGET", "EAST", "WEST", 5)

InventoryManager.transfer:
  Lock warehouses in consistent order (EAST < WEST alphabetically)

  synchronized(EAST)
    synchronized(WEST)
      EAST.removeStock("WIDGET", 5):
        currentQty = 7
        7 >= 5, sufficient stock
        inventory["WIDGET"] = 2
        return true

      removeStock succeeded, proceed:

      WEST.addStock("WIDGET", 5):
        currentQty = 0
        inventory["WIDGET"] = 5

  Release both locks

Result: EAST = 2, WEST = 5 (transfer complete, operation was atomic)
```

Both warehouses were locked throughout the entire operation. No other thread could read from EAST or WEST during the transfer, so there's no observable state where the 5 units are missing from both. The lock ordering by warehouse ID prevents deadlock if another thread tries to transfer in the opposite direction.

Test Case 3: Validation Prevents Negative Inventory

When testing validation, we're wanting to verify that we properly reject operations that would violate the "no negative inventory" invariant.

Setup: EAST has 2 widgets, WEST has 5 widgets

```
Operation: transfer("WIDGET", "EAST", "WEST", 10)

InventoryManager.transfer:
  synchronized(EAST)
    synchronized(WEST)
      EAST.removeStock("WIDGET", 10):
        currentQty = 2
        2 < 10, insufficient stock
        return false

      removeStock returned false, abort transfer:
      return false

Result: EAST still has 2, WEST still has 5 (no state changed)

Operation: getWarehousesWithAvailability("WIDGET", 4)

  Check EAST: 2 < 4, return false
  Check WEST: 5 >= 4, return true

  return ["WEST"]

Result: Only WEST can fulfill a 4-unit order
```

The transfer correctly rejected the operation because EAST only had 2 units. Even though we locked both warehouses, we validated before making any changes, so both warehouses are unchanged. Then `getWarehousesWithAvailability` correctly identified that only WEST can fulfill a 4-unit order.

These three cases verify the core invariants: alerts fire correctly without duplicates, transfers are atomic and deadlock-free, and we never allow negative inventory.

Extensibility

If there's time after implementation, interviewers often ask "what if" questions to test whether your design can evolve. You typically won't implement these changes, you'll just explain where they'd fit.

For this problem in particular, it's unlikely there is time to go into anything other than what we touched on above. If, you crushed it, the most natural extension is to add a reservation system to prevent overselling. Let's talk about what it would look like.

"How do you prevent overselling when orders are in progress?"

The current system works correctly from a correctness standpoint in that the locking prevents actual overselling. However, it creates a bad user experience. In e-commerce, "fulfilling an order" isn't instant. There's a window between "customer clicks buy" and "inventory is deducted." During checkout, the customer enters shipping info, payment details, maybe applies a coupon. That takes 30-60 seconds.

If we only check and deduct inventory when payment succeeds, customers can have a frustrating experience. Imagine two customers both see "1 item available" for a product. Both click "buy now" and start entering their credit card information. After 45 seconds of typing in payment details, one customer submits first and the order succeeds. The second customer hits submit 10 seconds later, and now they get an error: "Sorry, this item is out of stock." They just wasted a minute filling out a form for nothing.

> "The solution is to add a reservation system. When a customer starts checkout, we reserve inventory without removing it. Reserved inventory can't be allocated to other orders, but it hasn't left the warehouse yet. If the customer completes checkout within a timeout window, we confirm the reservation and actually remove the stock. If they abandon the cart or the timer expires, we release the reservation back into the available pool."

This approach prioritizes fairness over conversion optimization. It's great for high-demand scenarios (limited edition drops, concert tickets, high-value items) where you want to prevent customers from wasting time on checkout flows that can't succeed. Many high-volume retailers (Amazon, Walmart) actually prefer optimistic strategies instead. They accept orders even with uncertain inventory, then handle stockouts through substitutions or refunds. That maximizes sales but creates occasional customer service friction.

> Regardless, its a common follow up!

Track reserved quantities separately from available inventory. When checking availability, consider both.

WAREHOUSE WITH RESERVATIONS

```
class Warehouse:
    - id: string
    - inventory: Map<string, int>
    - reserved: Map<string, int>
    - reservations: Map<string, Reservation>
    - alertConfigs: Map<string, List<AlertConfig>>

    + reserveStock(productId, quantity, reservationId, timeoutMs) -> boolean
    + confirmReservation(reservationId) -> boolean
    + releaseReservation(reservationId) -> void

class Reservation:
    - productId: string
    - quantity: int
    - expiresAt: long
```

The `inventory` map still tracks physical stock in the warehouse. The new `reserved` map tracks how much is currently held for pending orders. When checking availability, we compute `available = inventory[productId] - reserved[productId]`.

WAREHOUSE.CHECKAVAILABILITY WITH RESERVATIONS

```
checkAvailability(productId, quantity)
    synchronized(this)
        if quantity <= 0
            return false

        totalQty = inventory[productId] ?: 0
        reservedQty = reserved[productId] ?: 0
        availableQty = totalQty - reservedQty

        return availableQty >= quantity
```

When a customer starts checkout, we call `reserveStock`:

WAREHOUSE.RESERVESTOCK

```
reserveStock(productId, quantity, reservationId, timeoutMs)
    synchronized(this)
        totalQty = inventory[productId] ?: 0
        reservedQty = reserved[productId] ?: 0
        availableQty = totalQty - reservedQty

        if availableQty < quantity
            return false

        // Create reservation record
        reservation = Reservation(productId, quantity, currentTime() + timeoutMs)
        reservations[reservationId] = reservation

        // Update reserved count
        reserved[productId] = reservedQty + quantity
        return true
```

The reservation gets stored with its expiration time. If payment succeeds, we confirm:

WAREHOUSE.CONFIRMRESERVATION

```
confirmReservation(reservationId)
    synchronized(this)
        reservation = reservations[reservationId]
        if reservation == null
            return false

        if currentTime() > reservation.expiresAt
            return false  // Reservation expired

        // Actually deduct inventory
        inventory[reservation.productId] -= reservation.quantity

        // Free up the reserved count
        reserved[reservation.productId] -= reservation.quantity

        // Remove reservation record
        reservations.remove(reservationId)
        return true
```

If checkout is abandoned or times out, we release:

WAREHOUSE.RELEASERESERVATION

```
releaseReservation(reservationId)
    synchronized(this)
        reservation = reservations[reservationId]
        if reservation == null
            return

        reserved[reservation.productId] -= reservation.quantity
        reservations.remove(reservationId)
```

This prevents the bad user experience because reserved inventory is invisible to other customers. Two customers can't reserve the same unit. The first customer to click "buy now" gets the reservation, and the second customer immediately sees "out of stock" instead of wasting time filling out a checkout form.

But then how do we expire reservations?

You need a background cleanup task to handle expired reservations. If a customer abandons their cart without explicitly releasing the reservation, that inventory stays locked forever. The cleanup task periodically scans reservations, finds expired ones, and calls `releaseReservation`.

CLEANUP TASK

```
cleanupExpiredReservations()
    while true
        sleep(60000)  // Run every minute
        now = currentTime()

        synchronized(this)
            for reservationId in reservations.keys()
                reservation = reservations[reservationId]
                if now > reservation.expiresAt
                    releaseReservation(reservationId)
```

The timeout value matters. Too short (30 seconds) and you cancel legitimate checkouts. Too long (10 minutes) and you keep inventory locked while customers browse. Most e-commerce

sites use 5-15 minutes.

I'd probably note to my interviewer at this stage that tuning the timeout will be a business decision that trades off unhappy customers who lost their reservation and unhappy customers who didn't get a chance to buy. Most businesses will try to maximize revenue, which usually means having a grace period where slightly-expired reservations can still be confirmed if payment is in progress.

"How would you handle inventory that's being shipped between warehouses?"

The atomic transfer we implemented earlier is simplified for interview scope. It assumes inventory magically teleports between warehouses. In reality, when you transfer stock from California to New York, those units spend 3-5 days on a truck. They're not available at either location during that time, but they haven't disappeared from the system either.

One elegant solution is to treat a Transfer as a place where inventory can temporarily live, just like a Warehouse. Both implement the same interface for holding inventory.

INVENTORYHOLDER INTERFACE

```
interface InventoryHolder:
    + addStock(productId, quantity) -> void
    + removeStock(productId, quantity) -> boolean
    + getStock(productId) -> int
    + checkAvailability(productId, quantity) -> boolean
```

Both `Warehouse` and `Transfer` implement this interface. A warehouse holds inventory long-term across many products. A transfer holds inventory temporarily for a single product during shipment.

TRANSFER AS INVENTORYHOLDER

```
class Transfer implements InventoryHolder:
    - id: string
    - productId: string
    - quantity: int
    - fromWarehouseId: string
    - toWarehouseId: string
    - createdAt: timestamp
```

```
    + Transfer(id, productId, quantity, fromWarehouseId, toWarehouseId)
    + getStock(productId) -> int
    + getFromWarehouse() -> string
    + getToWarehouse() -> string
```

When you initiate a transfer, you remove stock from the source warehouse and "add" it to a Transfer object:

INITIATING A TRANSFER

```
initiateTransfer(productId, fromWarehouseId, toWarehouseId, quantity)
    fromWarehouse = warehouses[fromWarehouseId]
    toWarehouse = warehouses[toWarehouseId]

    if !fromWarehouse.removeStock(productId, quantity)
        return null  // Insufficient stock

    // Create transfer to hold the inventory during shipment
    transfer = Transfer(generateId(), productId, quantity, fromWarehouseId, toWare
    transfers[transfer.id] = transfer

    return transfer.id
```

The inventory now lives in the Transfer object. It's not available at either warehouse, but it hasn't vanished from the system. You can query total system inventory by summing across all warehouses and all transfers.

When the shipment arrives (maybe an external tracking system calls your API), you complete the transfer:

COMPLETING A TRANSFER

```
completeTransfer(transferId)
    transfer = transfers[transferId]
    if transfer == null
        return false

    toWarehouse = warehouses[transfer.toWarehouseId]

    // Move inventory from transfer to destination
```

```
    toWarehouse.addStock(transfer.productId, transfer.quantity)

    // Remove the transfer object
    transfers.remove(transferId)
    return true
```

This design keeps inventory fully accounted for. At any moment, you can sum across warehouses and transfers to get total system inventory. In-transit stock doesn't count as available for new orders (only warehouse stock does), but you can query where every unit is. If a shipment needs to be cancelled, you just return the quantity to the source warehouse.

The 'aha moment' is that making `Transfer` a first-class entity that can hold inventory gives you a much more accurate model of how physical inventory systems actually work. This is better than treating transfer as just an operation.

What is Expected at Each Level?

This one was a little more complex than some of the other breakdowns, so what is actually expected at each level?

Junior

At the junior level, I'm checking whether you can build a working system that tracks inventory correctly. You should identify the need for an `InventoryManager` to coordinate operations and `Warehouse` to hold stock quantities. Your `addStock` and `removeStock` methods should update quantities correctly and validate that stock doesn't go negative. Basic error handling matters: reject operations that would create negative inventory, handle invalid warehouse IDs. You might not recognize the concurrency issues on your own, but that's fine at this level. If I ask "what happens if two threads try to remove stock at the same time?", it's okay to need hints about locks or synchronization. The key is demonstrating you can implement the basic operations and enforce the "no negative stock" rule.

Mid-level

For mid-level candidates, I expect cleaner separation of concerns and awareness of concurrency issues, though you might need some guidance on the subtle edge cases.

`InventoryManager` should orchestrate cross-warehouse operations, while each `Warehouse` manages its own inventory map and alerts. You should recognize that concurrent operations need synchronization and implement basic locking (synchronized methods or equivalent). I expect you to handle the transfer operation carefully - checking that you validate stock availability, but you might not immediately see the time-of-check-time-of-use race condition without a hint. The alert system should use the `AlertListener` interface to decouple notification logic.

Senior

Senior candidates should produce a design that handles concurrency correctly without prompting. You should proactively discuss the race conditions in `transfer` and explain why you need to lock both warehouses atomically with proper ordering to prevent deadlock. You should recognize the time-of-check-time-of-use bug in the naive transfer implementation and explain why checking `removeStock` 's return value isn't enough - you need atomic operations across both warehouses. I expect you to discuss trade-offs between coarse-grained (lock entire warehouse) vs fine-grained (lock per product) synchronization, explaining when each makes sense.